Results & Logs Page — Info Doc

Page Overview

Purpose

The Results & Logs page is the historical ledger for every execution produced by `resmon`. It lets the user browse a unified table of local and (when signed in) cloud executions, open any row to read its Markdown report, inspect the per-run log, examine execution metadata, and replay structured progress events. From the same surface the user can bulk-export selected local executions as a single zipped bundle and bulk-delete selected local executions.

Primary User Flows

1. Browse every execution in reverse-chronological order (Local + Cloud merged).
2. Filter the table by execution type (deep dive / deep sweep / automated sweep), status, and source (All / Local / Cloud).
3. Click a row to open the viewer and switch between the Report, Log, Metadata, and Progress tabs.
4. Select one or more local rows and export them to disk as a zip bundle, optionally revealing the output in Finder / File Explorer.
5. Select one or more local rows and delete them after a confirmation modal.
6. Deep-link into a specific execution (and optional tab) via URL hash `#…exec=<id>…tab=<report|log|meta|progress>`.

Inputs and Outputs

- **Inputs:** no free-form input fields on the page itself. Filter chips (type, status, location), row checkboxes, row clicks, and the URL hash drive all behaviour. `GET /api/executions/merged?filter=<all|local|cloud>&limit=200` supplies the table rows.
- **Outputs:** rendered Markdown report, raw log text, metadata JSON, and progress-event list from the selected execution; a zip file on disk for exports (path returned and displayed to the user); deletion of local execution rows.

Known Constraints or Permissions

- Cloud rows are read-only from this page: they can be viewed but cannot be selected for export or delete. `handleToggleAll` skips any row whose `execution_location` is not `local`, and the row-click branch routes cloud rows to an informational cloud card instead of the full `ReportViewer`.
- The Local / Cloud / All filter chip is only rendered when `useAuth().isSignedIn` is true (`showLocationFilter={isSignedIn}`).
- Cloud artifacts are not streamed through this backend: the cloud card simply explains that artifacts are fetched on demand and cached under `~/Library/Application Support/resmon/cloud-cache/`.
- The export flow depends on the Storage tab's `export_directory` setting when set; otherwise a temporary file is used.
- "Reveal in Finder" / "Reveal in File Explorer" is only shown when the Electron preload exposes `window.resmonAPI.revealPath`; the label switches on `window.resmonAPI.platform === 'darwin'`.

Frontend

Route and Main Component

- Route: `/results` (mounted from `App.tsx`).
- Main component: resmon_scripts/frontend/src/pages/ResultsPage.tsx.

Child Components and Hooks

- `ResultsList` — renders the filterable/selectable table, filter chips, and row-click dispatch.
- `ReportViewer` (resmon_scripts/frontend/src/components/Results/ReportViewer.tsx) — the four-tab viewer mounted for local rows.
- `PageHelp` — the collapsible in-page help panel with "The table", "Viewing a report", and "Exporting" sections.
- `useExecutionsMerged('all', 200)` (resmon_scripts/frontend/src/hooks/useExecutionsMerged.ts) — fetches `/api/executions/merged?filter=…&limit=…`, exposes `executions`, `filter`, `setFilter`, `loading`, `error`, `refresh`, and auto-refreshes on the `completionCounter` from `ExecutionContext` and on the `resmon:cloud-sync-applied` window event.
- `useExecution()` — supplies `completionCounter`, which the page re-fires into `refresh()` via a `useEffect` so rows that just finished appear without a manual reload.

- `useAuth()` — exposes `isSignedIn`, gating the Local / Cloud / All filter chip.
- `apiClient` — thin fetch wrapper used for `POST /api/executions/export` and `DELETE /api/executions/{id}`.
- `window.resmonAPI` (Electron preload) — `revealPath(path)` and `platform` power the reveal button and its label.

UI State Model

Local state in `ResultsPage`:

- `selected: Set<number>` — ids of checked rows (local-only; cloud rows are filtered out before selection can occur).
- `viewId: number | null` — currently open local execution; drives the `ReportViewer` card.
- `viewCloudId: string | null` — currently open cloud execution UUID; drives the informational cloud card.
- `viewTab: 'report' | 'log' | 'meta' | 'progress' | undefined` — initial tab for `ReportViewer`, seeded from the URL hash.
- `typeFilter: string`, `statusFilter: string` — forwarded to `ResultsList` for client-side filtering.
- `error: string`, `exportPath: string`, `confirmDelete: boolean` — transient UI state for the error banner, export success banner (cleared after 10 s via `setTimeout`), and the delete-confirmation modal.
- `reportRef: RefObject<HTMLDivElement>` — scrolled into view (via `requestAnimationFrame(() => scrollIntoView({ behavior: 'smooth', block: 'start' }))`) whenever a row is opened.
- From `useExecutionsMerged`: `executions`, `locationFilter` (`filter`), `setLocationFilter` (`setFilter`), `loading`, `fetchError` (`error`), `refresh`.

Key Interactions and Events

- **Row toggle (`handleToggle`)**: flips the row's id in `selected`.
- **Toggle all (`handleToggleAll`)**: computes the currently visible, *local-only* filtered subset (skipping `execution_location !== 'local'`, plus any active `typeFilter` / `statusFilter`); if every filtered row is already selected, clears the selection; otherwise selects all of them.
- **Row click**: if the row's `execution_location` is `'cloud'`, sets `viewCloudId = execution_id` (clears `viewId`); otherwise sets `viewId = id` (clears `viewCloudId`).
- **Export (`handleExport`)**: `POST /api/executions/export` with `{ ids: Array.from(selected) }`; stores `resp.path` in `exportPath` and auto-clears it after 10 s. Errors populate `error`.

- **Reveal (`handleReveal`)**: invokes `window.resmonAPI.revealPath(exportPath)` ; label is "Reveal in Finder" on macOS (`platform === 'darwin'`) and "Reveal in File Explorer" otherwise.
- **Delete (`handleDeleteSelected`)**: iterates `selected` , issuing `DELETE /api/executions/<id>` per row (individual failures are swallowed to keep the batch going), clears the selection, dismisses the modal, and calls `refresh()` .
- **Deep link**: on mount, `window.location.hash` is parsed with `/exec=(\d+)/` to seed `viewId` , and `/tab=(report|log|meta|progress)/` to seed `viewTab` .
- **Auto-refresh on completion**: a `useEffect` keyed on `completionCounter` calls `refresh()` so locally-dispatched executions appear as soon as `ExecutionContext` marks them complete.
- **Cloud-sync refresh**: `useExecutionsMerged` subscribes to the `resmon:cloud-sync-applied` window event and refetches whenever `useCloudSync` signals that a new page of cloud rows was ingested.
- **Name column.** The table renders a `Name` column whose cell prefers the row's `saved_configuration_name` (joined in by `_enrich_execution_row` from `saved_configurations` via the `executions.saved_configuration_id` foreign key), falling back to the `routine_name` for routine-fired rows, and finally to `Execution #{e.id}` when neither is available. Manual executions launched from a loaded saved config (Deep Dive / Deep Sweep `ConfigLoader`) and "Save as new config" actions both populate `saved_configuration_id` , so the configured name surfaces here without an extra fetch.

Inside `ReportViewer` :

- Tabs are `Report` , `Log` , `Metadata` , and `Progress` ; the `Progress` tab label renders a pulsing indicator when the viewer's execution id matches the currently running execution (`activeExecution.executionId === executionId && status === 'running'`).
- On mount and whenever `executionId` changes, it clears stale state and fires three parallel fetches:
  - `GET /api/executions/{id}` → `meta`
  - `GET /api/executions/{id}/report` → `report_text` → `report`
  - `GET /api/executions/{id}/log` → `log_text` → `log`
- **Progress tab contract**: when the execution is live (`isLive`), events are mirrored from `ExecutionContext.activeExecution.events` (the same buffer the Monitor page and floating widget consume). When historical, it fetches `GET /api/executions/{id}/progress/events` and renders that array. The backend route returns the live `progress_store` events if the execution is still registered in memory and falls back to persisted events from the database after cleanup.
- The viewer has its own single-row Export button that calls `POST /api/executions/export` with `{ ids: [executionId] }` and the same reveal behaviour as the page-level export.

Error and Empty States

- Loading: the page returns a `"Loading executions…"` placeholder while `useExecutionsMerged` is fetching.
- Fetch error: `fetchError` from the hook is rendered in a `.form-error` banner below the header (only when there is no page-level `error`).
- Page-level error: set by a failing export or other interactions and rendered in a `.form-error` banner.
- Report / Log absent: `ReportViewer` shows "No report available." / "No log available." when the corresponding fetch returns 404 or the fields are null.
- Selected but no action possible: the Export Selected and Delete Selected buttons are disabled while `selected.size === 0`.
- Cloud row opened: renders an informational card explaining that cloud artifacts are fetched on demand and cached locally, exposing the cloud execution UUID in a `<code>` block.
- Delete confirmation: the confirmation modal is the only way deletion proceeds; clicking the overlay dismisses it.

Backend

API Endpoints

All paths live in resmon_scripts/resmon.py and are served by the local FastAPI daemon.

- `GET /api/executions/merged?filter=<all|local|cloud>&limit=<int>` — merged local + cloud-mirror list, registered before the `{exec_id}` route so the literal `merged` segment wins over the int path parameter.
- `GET /api/executions/{exec_id}` — full row for one local execution.
- `GET /api/executions/{exec_id}/report` — returns `{ "report_text": "…" }` read from `row["result_path"]`.
- `GET /api/executions/{exec_id}/log` — returns `{ "log_text": "…" }` read from `row["log_path"]`.
- `GET /api/executions/{exec_id}/progress/events` — returns live events from `progress_store` if the execution is still registered, otherwise the persisted events from the database. Sends `Cache-Control: no-store, no-cache, must-revalidate` and `Pragma: no-cache` to defeat Chromium's heuristic cache.
- `GET /api/executions/{exec_id}/progress/stream` — SSE stream for live execution progress (used by `ExecutionContext` / Monitor; the `ReportViewer` Progress tab consumes the events indirectly through `ExecutionContext.activeExecution.events` while live).

- `POST /api/executions/{exec_id}/cancel` — cooperative cancellation; 409 if the execution is not currently active in `progress_store`.
- `DELETE /api/executions/{exec_id}` — deletes the row from the `executions` table (cascades through `execution_documents` per schema).
- `POST /api/executions/export` — bundles the selected executions into a zip and returns `{ "path": "…", "count": N }`.
- Cloud-mirror cache endpoints used by the companion `useCloudSync` flow:
  - `GET /api/cloud-sync/executions?limit=<int>` — cloud-mirror executions (newest-first).
  - `POST /api/cloud-sync/cache/record` — records a freshly downloaded cloud artifact (`execution_id`, `artifact_name`, `local_path`, `bytes`) and evicts LRU entries if the cache exceeds `max_bytes` (defaults to `CLOUD_CACHE_MAX_BYTES_DEFAULT`).
  - `POST /api/cloud-sync/cache/touch` — bumps `last_accessed_at` on a cached entry (keeps it hot in LRU).
  - `GET /api/cloud-sync/cache/{execution_id}/{artifact_name}` — returns cache metadata for a specific artifact; 404 if not cached.

Request and Response Patterns

- `GET /api/executions/merged`: validates `filter ∈ {all, local, cloud}` (400 on mismatch); clamps `limit` to `[1, 1000]`. When `filter` includes `local`, it reads `get_executions(conn, limit=limit)` and tags each row with `execution_location = "local"`. When it includes `cloud`, it reads `get_cloud_executions(conn, limit=limit)` and tags rows with `execution_location = "cloud"`, backfills `start_time` / `end_time` from `started_at` / `finished_at`, and sets `execution_type = "cloud_routine"`. The combined list is sorted by `start_time` (falling back to `started_at`) descending, then truncated to `limit`.
- Every local row is passed through `_enrich_execution_row`, which decodes the JSON `parameters` column into `query`, `keywords`, and `repositories`, and aliases `result_count` / `new_result_count` to `total_results` / `new_results` so the frontend renders a single schema.
- `GET /api/executions/{exec_id}` / `/report` / `/log`: 404 when the row does not exist or when the file path stored on the row is missing/unreachable on disk.
- `POST /api/executions/export`: body is `ExecutionExport { ids: list[int] }`. Only local rows are resolved (cloud UUIDs are not valid here). 404 if none of the supplied ids match. On success the response is `{ "path": "<absolute zip path>", "count": <int> }`.
- `POST /api/executions/{exec_id}/cancel`: 409 `"Execution not running"` if `progress_store.is_active(exec_id)` is false; otherwise calls `progress_store.request_cancel(exec_id)` and returns `{ "status": "cancellation_requested" }`.

- `GET /api/executions/{exec_id}/progress/events` : 404 when the execution id is unknown; otherwise returns a JSON array of event objects sourced from `progress_store.get_events(exec_id)` (if registered) or `get_progress_events(conn, exec_id)` (persisted fallback).
- `GET /api/executions/{exec_id}/progress/stream` : returns an SSE stream (`text/event-stream`) with `event: progress` frames and monotonically increasing `id:` cursors. When the execution is already terminal (`completed | failed | cancelled`) and not in the live store, it replays persisted events as a one-shot batch starting from the client's `last_event_id` cursor and closes.

Persistence Touchpoints

- Local executions live in the `executions` table (resmon_scripts/implementation_scripts/database.py) with columns including `execution_type` , `routine_id` , `parameters` (JSON), `start_time` , `end_time` , `status` , `result_count` , `new_result_count` , `log_path` , `result_path` , `error_message` , `progress_events` , `current_stage` . `DELETE FROM executions WHERE id = ?` cascades to `execution_documents` via `ON DELETE CASCADE` .
- Cloud mirror lives in `cloud_executions` and `cloud_routines` , keyed by cloud UUIDs with a monotonic `version` column used by the cursor-based sync protocol. Artifact cache bookkeeping lives in `cloud_cache_meta` (`execution_id` , `artifact_name` , `local_path` , `bytes` , `downloaded_at` , `last_accessed_at`) with LRU eviction driven by `evict_cloud_cache_if_needed` against a byte ceiling (`max_bytes` from the request, or `CLOUD_CACHE_MAX_BYTES_DEFAULT`).
- Report and log artifacts are files on disk under `resmon_reports/` :
  - Markdown reports: `resmon_reports/markdowns/<report_filename>.md` (path stored in `executions.result_path` ; written by `SweepEngine`).
  - Per-execution logs: `resmon_reports/logs/<log_filename>.txt` (path stored in `executions.log_path` ; written by the per-task logger).
  - Derived artifacts (figures, LaTeX, PDFs) live under `resmon_reports/figures/` , `resmon_reports/latex/` , and `resmon_reports/pdfs/` and are included opportunistically by `report_exporter.export_report_bundle` during export.
- The export destination directory is read from `get_setting(conn, "export_directory")` . When unset, the backend writes to a `tempfile.NamedTemporaryFile(suffix=".zip")` path.
- Persisted progress events are stored on the execution row (JSON in `progress_events`) and surfaced via `get_progress_events` when the in-memory `progress_store` entry has been cleaned up.

Execution Side Effects

- **Export (`POST /api/executions/export` → `_build_execution_zip`)**: creates a temporary staging directory, and for each selected execution writes an `execution_<id>/` folder containing: the Markdown report (copied from `result_path`), any bundled artifacts generated by `report_exporter.export_report_bundle` (e.g., compiled PDF and copied figures, guarded with a `try/except` that logs a warning on failure), a `metadata.json` snapshot of the row, and the log file (copied from `log_path`). A top-level `manifest.json` lists each execution's id, type, query, status, start/end times, and result counts. The staging directory is zipped (`ZIP_DEFLATED`) into the destination and the temp directory is removed on context exit. The same helper is reused for routine-completion email attachments.
- **Delete (`DELETE /api/executions/{exec_id}`)**: removes the row from `executions` and commits. On-disk report and log files are *not* removed by this endpoint — they remain under `resmon_reports/markdowns/` and `resmon_reports/logs/` until managed separately.
- **Cancel (`POST /api/executions/{exec_id}/cancel`)**: sets the cooperative cancel flag in `progress_store` ; the running sweep checks the flag at its yield points and transitions the execution to `status = 'cancelled'` with a terminal `Cancelled by user` progress event.
- **Cloud cache touch / record**: `record_cloud_cache_entry` inserts/updates a `cloud_cache_meta` row for a downloaded artifact; `evict_cloud_cache_if_needed` deletes oldest-accessed entries (and returns the evicted list) until the total `bytes` sum is within the configured ceiling. `touch_cloud_cache_entry` bumps `last_accessed_at` to keep an artifact hot.
- **Refresh fan-out**: `useExecutionsMerged` re-calls `GET /api/executions/merged` whenever `completionCounter` increments in `ExecutionContext` (local dispatch finished) or when the `resmon:cloud-sync-applied` window event fires from `useCloudSync` , keeping the Results table in step with both local and cloud state without user intervention.